

A Technical Introduction to RMSprop optimizer

Section 1

$$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2$$

$$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n \left(\hat{y}_i - y_i \right)^2$$

Section 2

Stochastic gradient descent (often abbreviated SGD) is an iterative method for optimizing an objective function with suitable smoothness properties (e.g. differentiable or subdifferentiable). It can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data). Especially in high-dimensional optimization problems this reduces the very high computational burden, achieving faster iterations in exchange for a lower convergence rate. The basic idea behind stochastic approximation can be traced back to the Robbins–Monro algorithm of the 1950s. Today, stochastic gradient descent has become an important optimization method in machine learning.

Section 3

$$Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2$$

$$Q(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2$$

where $m(w; x_i)$ is the predictive model (e.g., a deep neural network) the objective's structure can be exploited to estimate 2nd order information using gradients only. The resulting methods are simple and often effective

Section 4

Averaging Averaged stochastic gradient descent, invented independently by Ruppert and Polyak in the late 1980s, is ordinary stochastic gradient descent that records an average of its parameter vector over time. That is, the update is the same as for ordinary stochastic gradient descent, but the algorithm also keeps track of

Section 5

$$m_w(t) := \beta m_w(t-1) + (1 - \beta) \nabla W_L(t-1)$$

$$m_w(t) := \beta m_w(t-1) + (1 - \beta) \nabla W_L(t-1)$$

Section 6

$$w(t) := w(t-1) - \eta m_w(t) \vee w(t) + \epsilon$$

$$w(t) := w(t-1) - \eta \frac{\nabla W_L(t)}{\sqrt{\frac{1}{t} \sum_{i=1}^t \|\nabla W_L(i)\|^2}}$$

$$\{v\}_w^{(t)} + \epsilon$$

Section 7

A compromise between computing the true gradient and the gradient at a single sample is to compute the gradient against more than one training sample (called a "mini-batch") at each step. This can perform significantly better than "true" stochastic gradient descent described, because the code can make use of vectorization libraries rather than computing each step separately as was first shown in where it was called "the bunch-mode back-propagation algorithm". It may also result in smoother convergence, as the gradient computed at each step is averaged over more training samples. The convergence of stochastic gradient descent has been analyzed using the theories of convex minimization and of stochastic approximation. Briefly, when the learning rates η decrease with an appropriate rate, and subject to relatively mild assumptions, stochastic gradient descent converges almost surely to a global minimum when the objective function is convex or pseudoconvex, and otherwise converges almost surely to a local minimum. This is in fact a consequence of the Robbins–Siegmund theorem.

Section 8

History In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods, preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation")

was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

Section 9

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update Δw at each iteration, and determines the next update as a linear combination of the gradient and the previous update:

Section 10

This equation is implicit since w_{new} appears on both sides of the equation. It is a stochastic form of the proximal gradient method since the update can also be written as:

Section 11

$$v(w, t) := \gamma v(w, t-1) + (1 - \gamma) (\nabla Q_i(w))^2$$

$$\{ \text{displaystyle } v(w, t) := \gamma v(w, t-1) + \left(1 - \gamma \right) \left(\nabla Q_i(w) \right)^2 \}$$

Section 12

$$v_w(t) := \beta^2 v_w(t-1) + (1 - \beta^2) (\nabla_w L(t-1))^2$$

$$\{ \text{displaystyle } v_{\{w\}}^{\{t\}} := \beta_{\{2\}} v_{\{w\}}^{\{t-1\}} + \left(1 - \beta_{\{2\}} \right) \left(\nabla_{\{w\}} L^{\{t-1\}} \right)^2 \}$$